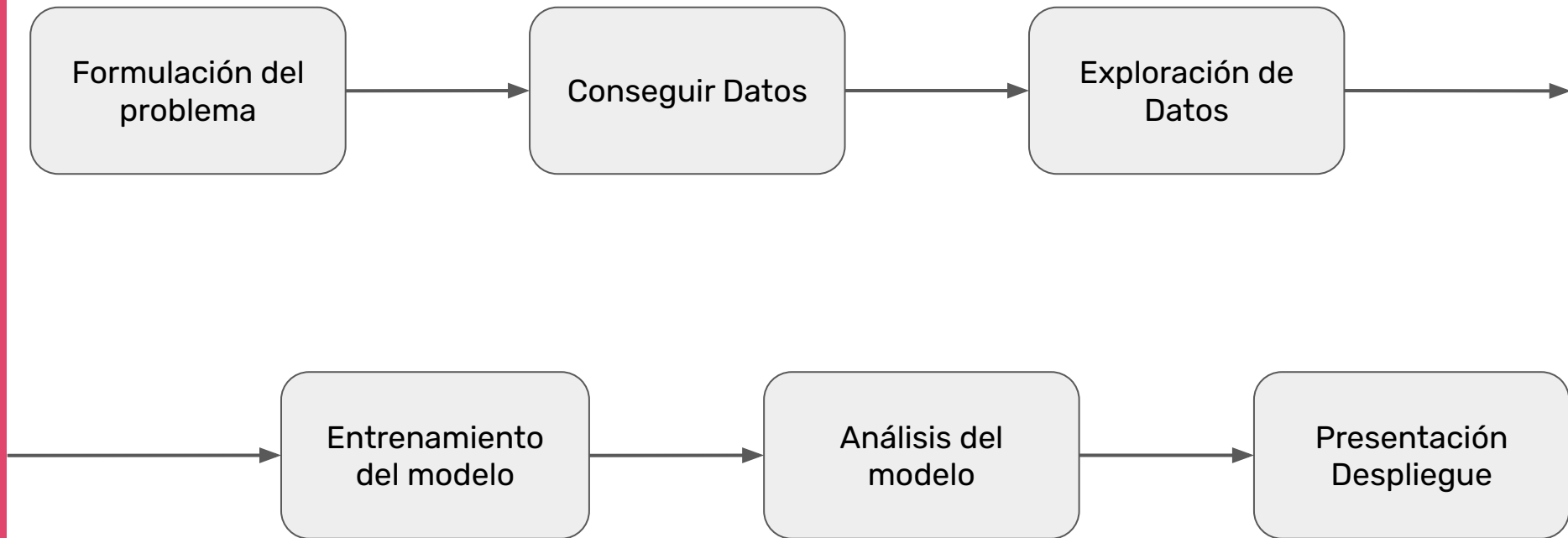

Introducción al Aprendizaje Automático

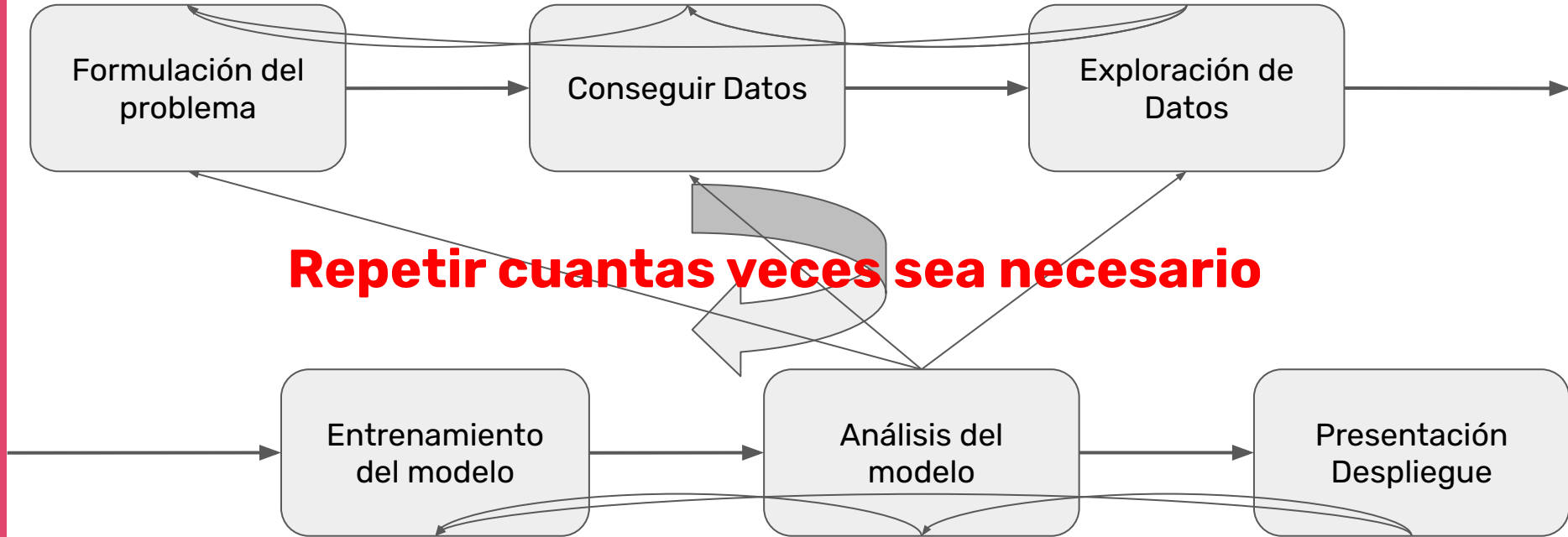
1er cuatrimestre 2025



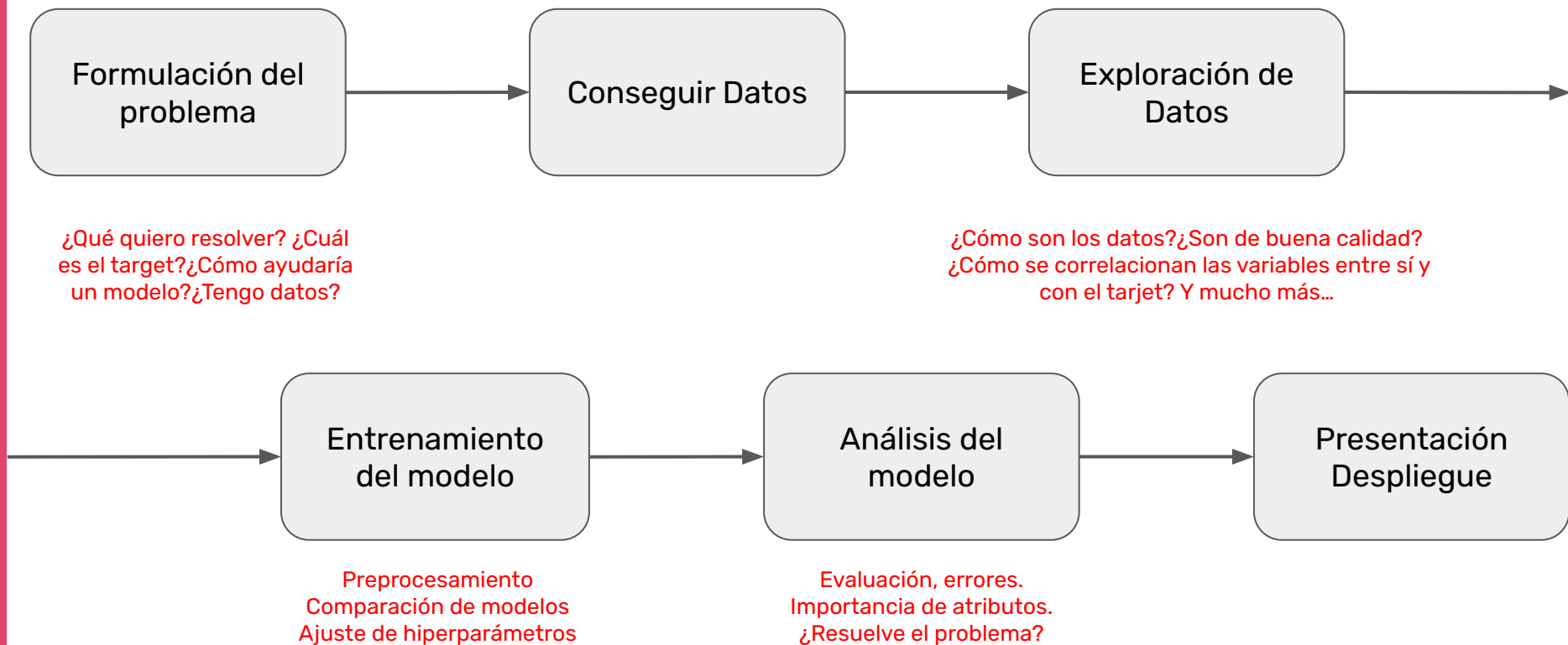
Ciclo de vida de un proyecto de Aprendizaje Automático



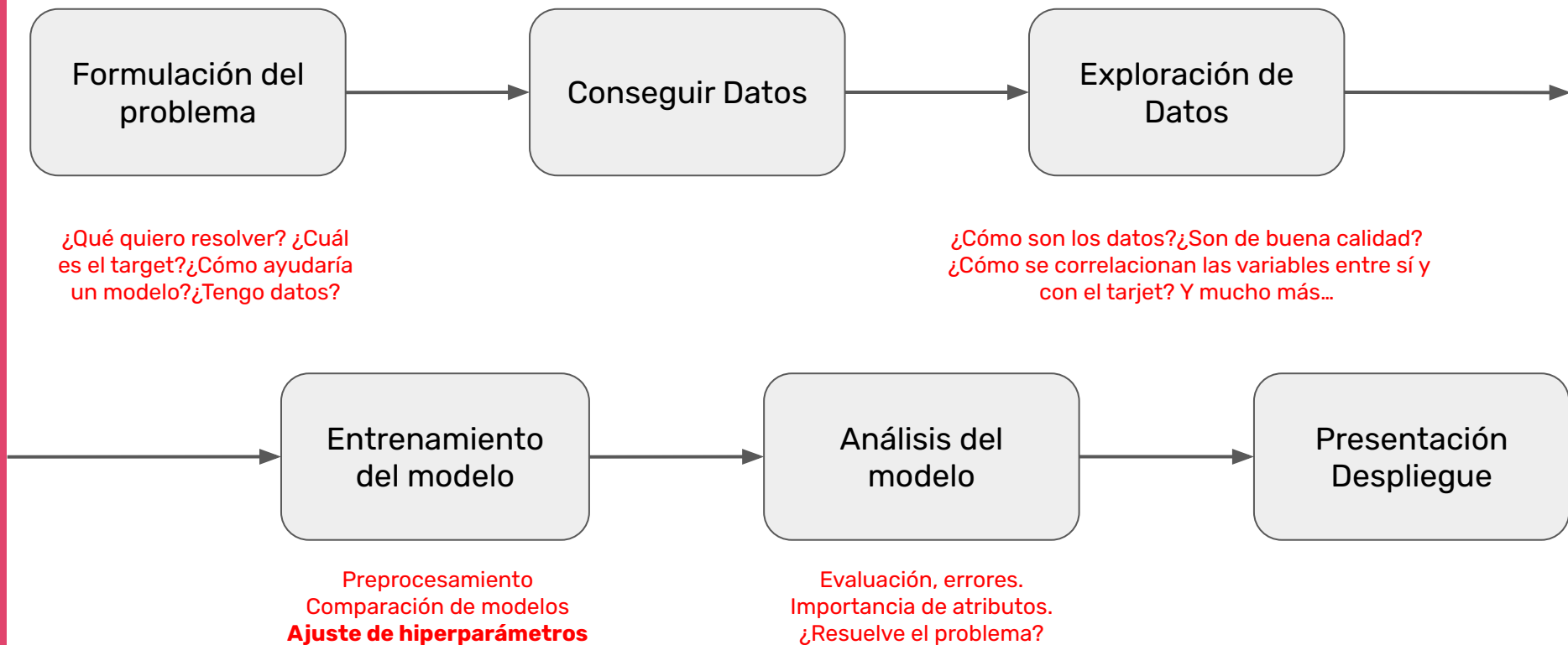
Ciclo de vida de un proyecto de Aprendizaje Automático



Ciclo de vida de un proyecto de Aprendizaje Automático



Ciclo de vida de un proyecto de Aprendizaje Automático



Optimización de Hiperparámetros

Antes de continuar, un comentario acerca de la nomenclatura...

PARÁMETRO

Son características de nuestro modelo que el algoritmo de entrenamiento aprende automáticamente. **Ejemplo:** pendiente y ordenada al origen en una regresión lineal, umbrales que usa un árbol de decisión para partir una rama, etc.

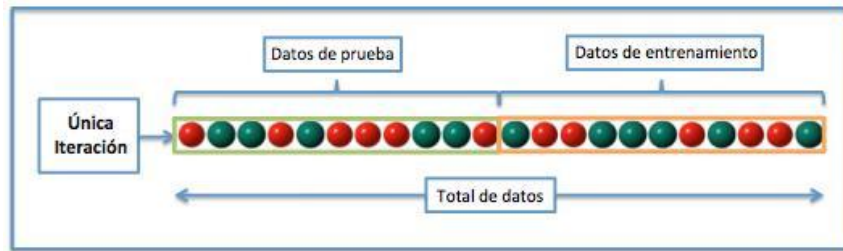
HIPERPARÁMETRO

Son características que debemos elegir ANTES bajo algún criterio. En general, el criterio es mejor performance estadística (¡aunque podría no serlo!).

En Scikit-Learn, los hiperparámetros los elegimos cuando creamos un modelo, mientras que podemos acceder a los parámetros recién luego de entrenar el modelo.

Repaso

Problema



Semilla 1

Modelo A

Exactitud: 0.76

Modelo B

Exactitud: 0.75

Semilla 2

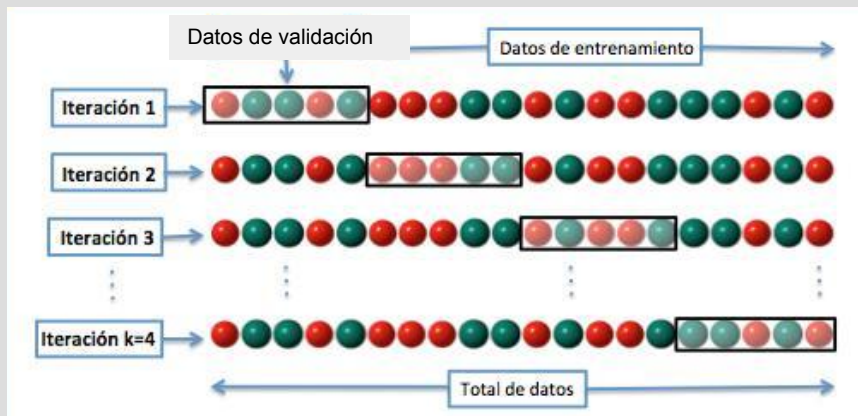
Modelo A

Exactitud: 0.74

Modelo B

Exactitud: 0.76

Solución: k-fold Cross Validation:



Modelo 1 → Performance 1

Modelo 2 → Performance 2

Modelo 3 → Performance 3

•
•

Promedio

¿Cómo elegimos los
mejores hiperparámetros
para nuestro problema?

¿Cómo elegimos los mejores hiperparámetros para nuestro problema?

¿Qué es mejor? ¿Con respecto a exactitud?
¿Área bajo la curva ROC?

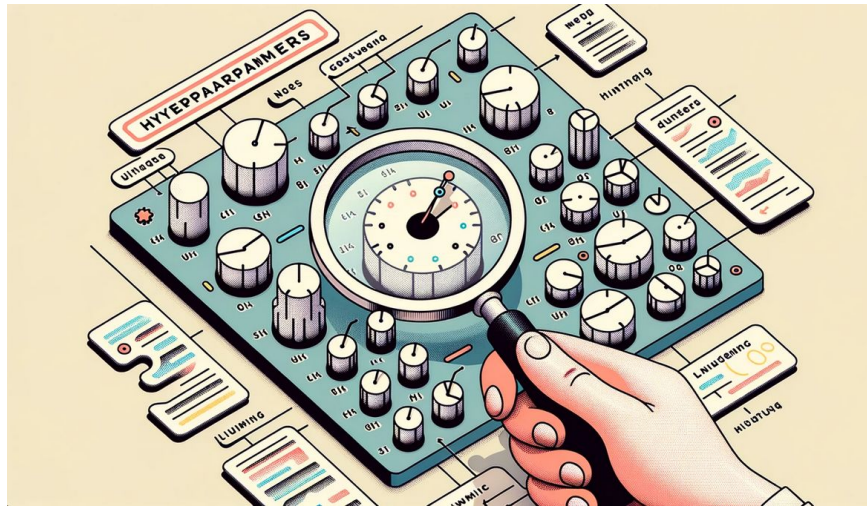
¿Cómo elegimos los mejores hiperparámetros para nuestro problema?

¿Qué es mejor? ¿Con respecto a exactitud?
¿Área bajo la curva ROC?

Primero, tenemos que definir una métrica a optimizar.

Una vez que sabemos la métrica que queremos optimizar,
¿cómo elegimos los mejores hiperparámetros para nuestro problema?

Solución 1: probar a mano distintos valores de los hiperparámetros



Una vez que sabemos la métrica que queremos optimizar,
¿cómo elegimos los mejores hiperparámetros para nuestro problema?

Solución 1: probar a mano distintos valores de los hiperparámetros

Cansador, tedioso y poco eficiente.

Una vez que sabemos la métrica que queremos optimizar,
¿cómo elegimos los mejores hiperparámetros para nuestro problema?

Solución 2: hacer una búsqueda exhaustiva. Es decir probando con todos los valores de los hiperparámetros que podamos y eligiendo la mejor combinación. Este método se llama **Grid Search** (“búsqueda de cuadrícula”).

Solución 2: hacer una búsqueda exhaustiva. Es decir probando con todos los valores de los hiperparámetros que podamos y eligiendo la mejor combinación. Éste método se llama Grid Search (“búsqueda de cuadrícula”).

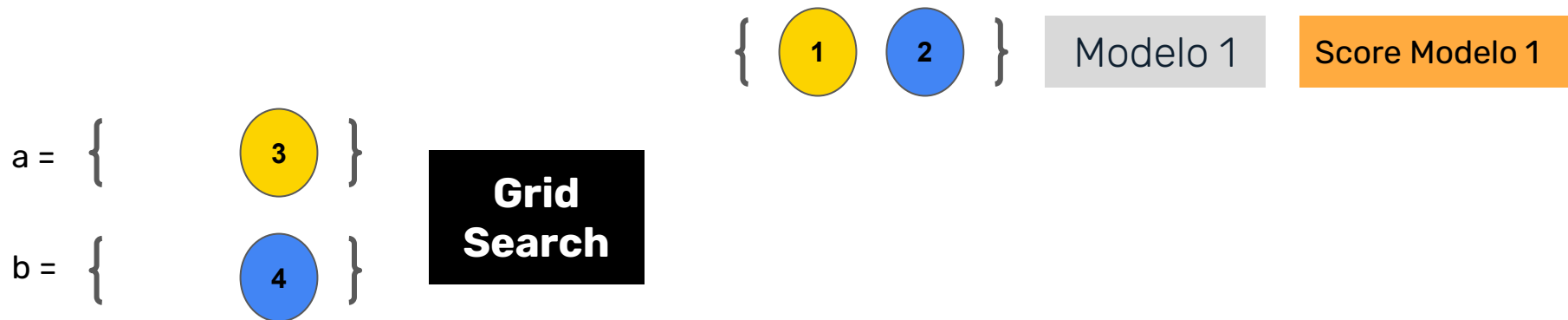
Por ejemplo, si tenemos dos hiperparámetros, a y b , que pueden tomar valores $a = \{1,3\}$ y $b = \{2,4\}$

$$a = \{ \text{1} \quad \text{3} \}$$

$$b = \{ \text{2} \quad \text{4} \}$$

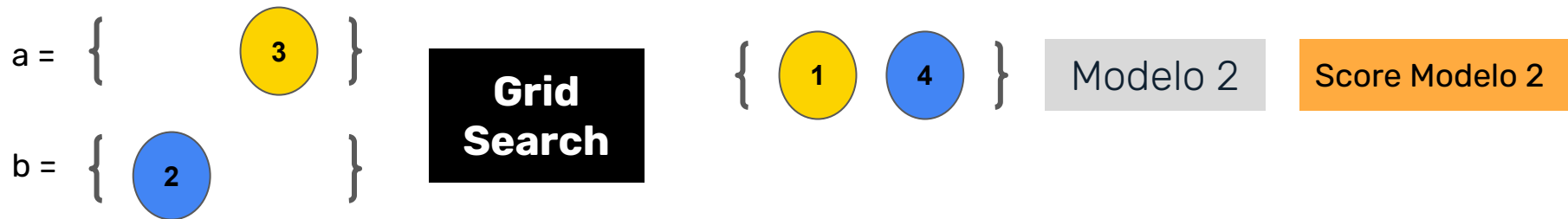
Solución 2: hacer una búsqueda exhaustiva. Es decir probando con todos los valores de los hiperparámetros que podamos y eligiendo la mejor combinación. Éste método se llama Grid Search (“búsqueda de cuadrícula”).

Por ejemplo, si tenemos dos hiperparámetros, a y b , que pueden tomar valores $a = \{1,3\}$ y $b = \{2,4\}$



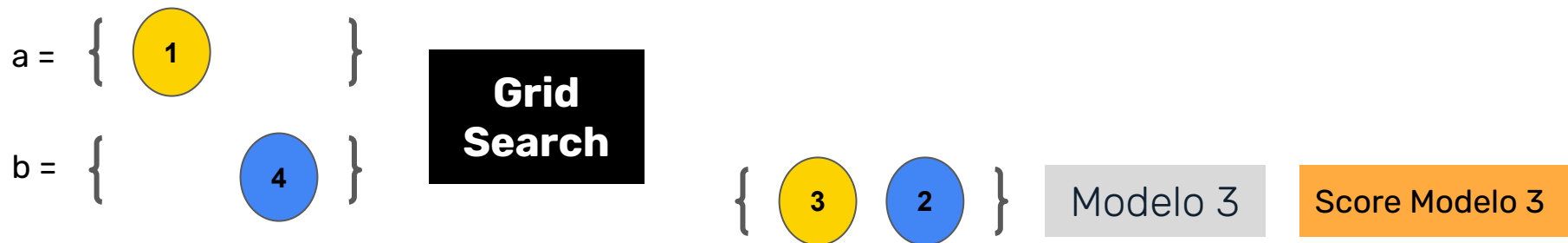
Solución 2: hacer una búsqueda exhaustiva. Es decir probando con todos los valores de los hiperparámetros que podamos y eligiendo la mejor combinación. Éste método se llama Grid Search (“búsqueda de cuadrícula”).

Por ejemplo, si tenemos dos hiperparámetros, a y b , que pueden tomar valores $a = \{1,3\}$ y $b = \{2,4\}$



Solución 2: hacer una búsqueda exhaustiva. Es decir probando con todos los valores de los hiperparámetros que podamos y eligiendo la mejor combinación. Éste método se llama Grid Search (“búsqueda de cuadrícula”).

Por ejemplo, si tenemos dos hiperparámetros, a y b , que pueden tomar valores $a = \{1,3\}$ y $b = \{2,4\}$



Solución 2: hacer una búsqueda exhaustiva. Es decir probando con todos los valores de los hiperparámetros que podamos y eligiendo la mejor combinación. Éste método se llama Grid Search (“búsqueda de cuadrícula”).

Por ejemplo, si tenemos dos hiperparámetros, a y b , que pueden tomar valores $a = \{1,3\}$ y $b = \{2,4\}$

$a = \{ \text{1} \}$
 $b = \{ \text{2} \}$

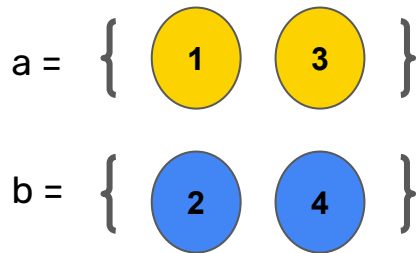
**Grid
Search**

$\{ \text{3} \text{ 4} \}$

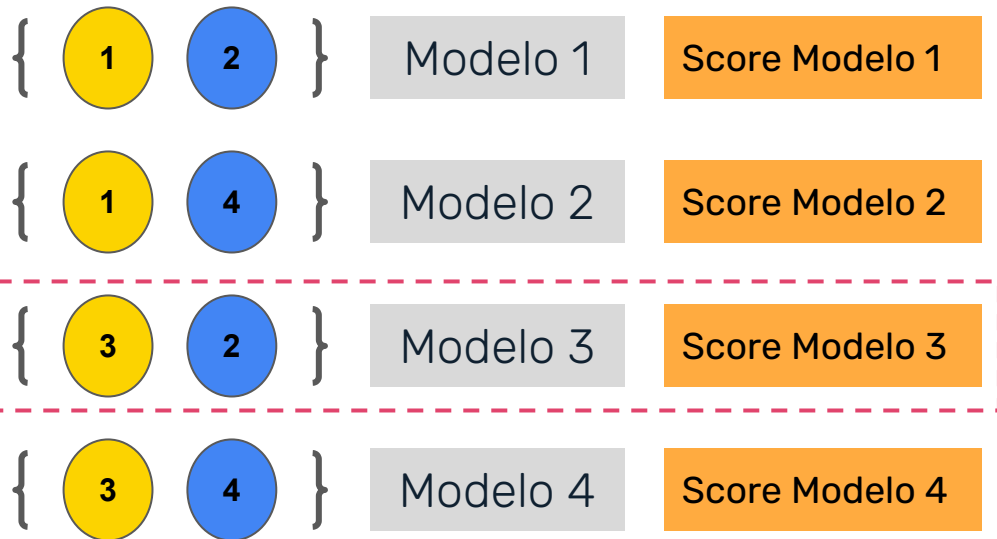
Modelo 4

Score Modelo 4

Elegimos el modelo con mejor score (o menor error si corresponde):



**Grid
Search**



Entonces, una vez que sabemos qué métrica queremos optimizar,

Grid Search consiste en:

1. Elegimos los valores que puede tomar cada hiperparámetro
2. Armamos las combinaciones “todos con todos” → Tenemos nuestra grilla
3. Recorremos la grilla, entrenamos un modelo para cada combinación y lo evaluamos.
4. Elegimos los hiperparámetros que definen el mejor modelo.

Entonces, una vez que sabemos qué métrica queremos optimizar,

Grid Search consiste en:

1. Elegimos los valores que puede tomar cada hiperparámetro
2. Armamos las combinaciones “todos con todos” → Tenemos nuestra grilla
3. Recorremos la grilla, entrenamos un modelo para cada combinación y **lo evaluamos.**
4. Elegimos los hiperparámetros que definen el mejor modelo.

¿Cómo evaluamos un modelo?

¿Con Train/Test split? ¿Con Validación Cruzada?

Al estar probando MUCHOS modelos, podría suceder que uno se desempeñe muy bien en el conjunto de entrenamiento simplemente por azar (incluso haciendo un nuevo train-test split dentro de ese conjunto).

Por esto, es muy importante evaluar cada modelo creado por Grid Search con Validación Cruzada en el conjunto de entrenamiento.

¿Cómo evaluamos un modelo?

¿Con Train/Test split? ¿Con Validación Cruzada?

Al estar probando MUCHOS modelos, podría suceder que uno se desempeñe muy bien en el conjunto de entrenamiento simplemente por azar (incluso haciendo un nuevo train-test split dentro de ese conjunto).

Por esto, es muy importante evaluar cada modelo creado por Grid Search con Validación Cruzada en el conjunto de entrenamiento.

¡Grid Search y Validación Cruzada suelen venir juntos!

¿Cómo establecemos el desempeño final del modelo luego de seleccionar los mejores hiperparámetros?

Algo de información del conjunto validación puede filtrarse en la elección de los hiperparámetros.

1. Elegimos los mejores hiperparámetros con Grid Search + Validación Cruzada
2. Entrenamos un modelo con esos hiperparámetros con todo el conjunto de entrenamiento
3. Evaluamos su performance con un conjunto de prueba reservado desde el inicio.

¿Qué desventajas tiene Grid Search?

¿Qué ocurre si tenemos, por ejemplo, cinco hiperparámetros y cinco valores para probar por hiperparámetro? ¿Qué tamaño tiene la grilla?

¿Qué desventajas tiene Grid Search?

¿Qué ocurre si tenemos, por ejemplo, cinco hiperparámetros y cinco valores para probar por hiperparámetro? ¿Qué tamaño tiene la grilla?

$5 \times 5 \times 5 \times 5 \times 5 = 3,125$ combinaciones



¿Qué desventajas tiene Grid Search?

¿Qué ocurre si tenemos, por ejemplo, cinco hiperparámetros y cinco valores para probar por hiperparámetro? ¿Qué tamaño tiene la grilla?

¿Y si, además, para cada modelo tenemos que hacer Validación Cruzada $k=10$?

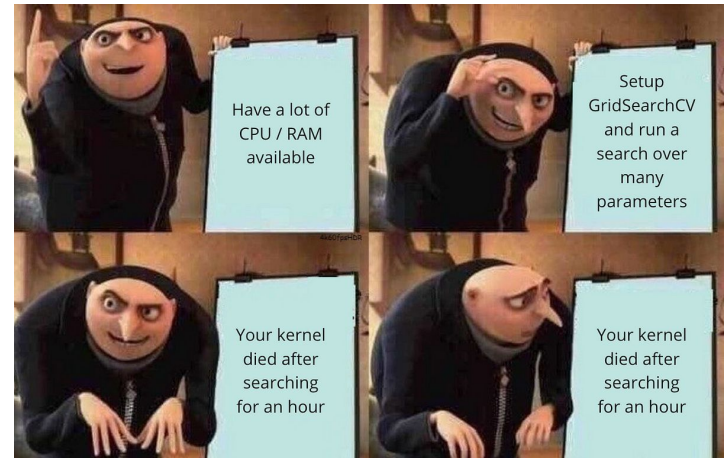
¿Qué desventajas tiene Grid Search?

¿Qué ocurre si tenemos, por ejemplo, cinco hiperparámetros y cinco valores para probar por hiperparámetro? ¿Qué tamaño tiene la grilla?

¿Y si, además, para cada modelo tenemos que hacer Validación Cruzada $k=10$?

$$3,125 \times 10 = 31,250$$

Costoso cuando el espacio de búsqueda es amplio



¿Qué desventajas tiene Grid Search?

Grid Search + CV puede ser computacionalmente muy demandante.

Al “probar” con pocos valores de los hiperparámetros nos estamos perdiendo algunas cosas importantes.

Además, suele haber hiperparámetros mucho más importantes que otros.

¿Qué desventajas tiene Grid Search?

Grid Search + CV puede ser computacionalmente muy demandante.

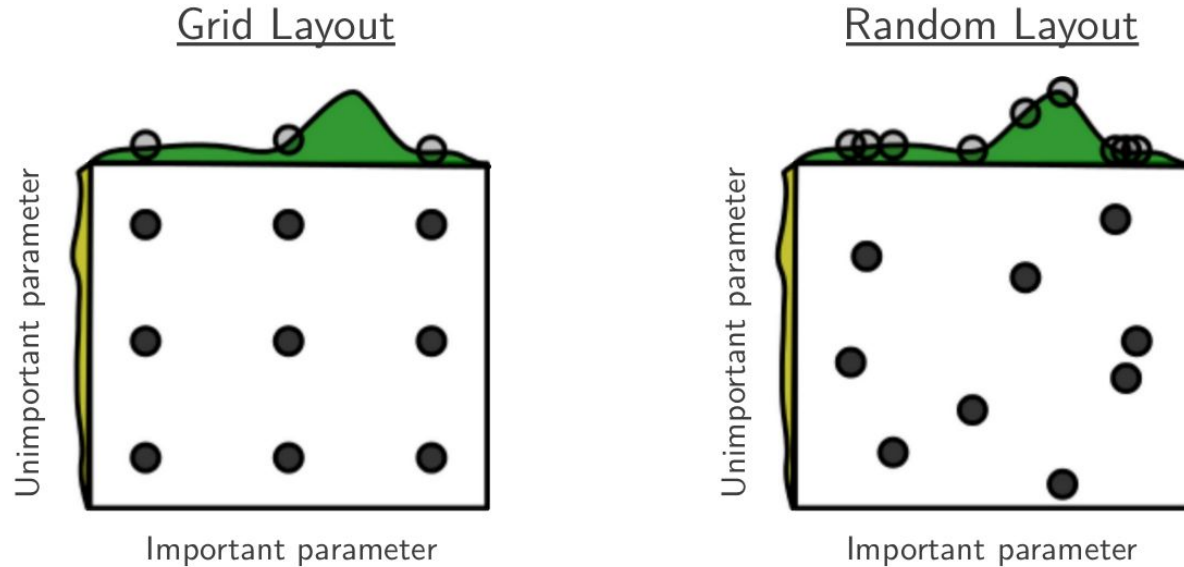
Al “probar” con pocos valores de los hiperparámetros nos estamos perdiendo algunas cosas importantes.

Además, suele haber hiperparámetros mucho más importantes que otros.

¿Y entonces?

Random Search explora opciones y combinaciones al azar, de manera menos “ordenada”. En muchas circunstancias, ¡esto es más eficiente, tanto desde el punto de vista de performance del modelo como de desempeño computacional!

[Fuente](#)



[Curse of dimensionality](#): Al generar puntos uniformemente en dimensiones altas, tanto el "medio" del hipercubo, como las esquinas están vacíos, y todo el volumen se concentra cerca de la superficie de una esfera de radio "intermedio" $d / 3$.

Resumen • **Para optimizar hiperparámetros necesitamos:**

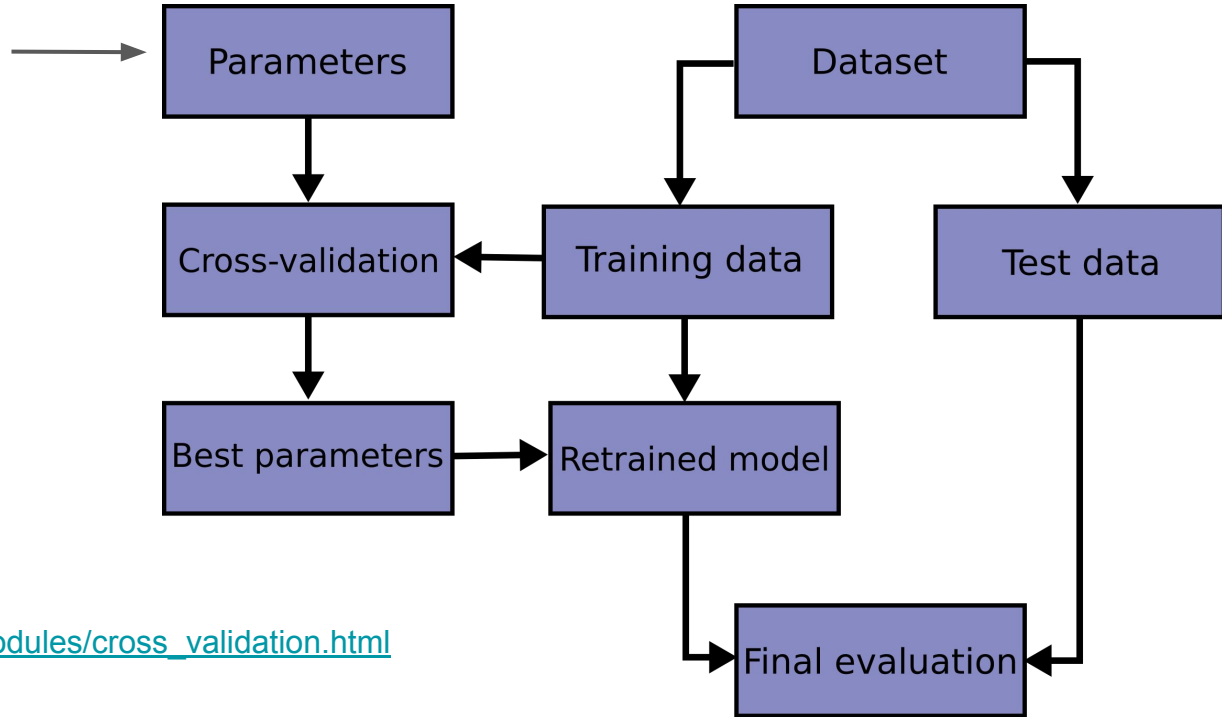
- Una **métrica** (exactitud, precisión, RMSE, ROC AUC, etc.)
- Un **modelo** (regresor o clasificador)
- Un **espacio** de (hiper)parámetros. Depende del tipo de modelo que estemos usando.
- Un **método** para buscar o muestrear los candidatos
 - a. **Grid Search**: Plantea opciones y explora todas las combinaciones
 - b. **Random Search**: explora opciones y combinaciones al azar.
 - c. **Otros**: Optimización Bayesiana, Enfoque iterativo. (Wiki: [Lista de algoritmos de optimización](#))
- Un **evaluación de desempeño final**

¡Una guía para ordenar el proceso!

1. Separar los datos en *train* (entrenamiento) y *test* (prueba).
2. Armar el Grid-Search, definiendo modelos y rangos de sus hiperparámetros para:
 - a. Preprocesamiento de features
 - b. Algoritmo de ML
3. Ejecutar el Grid-Search, evaluando con K-fold cross-validation la *performance* de cada modelo construido con cada una de todas las combinaciones posibles.
4. Elegir el modelo de mejor performance, y entrenarlo con todos los datos de entrenamiento.
5. Estimar la performance del modelo evaluando sobre *test* (evaluación).
6. El modelo sale a producción.

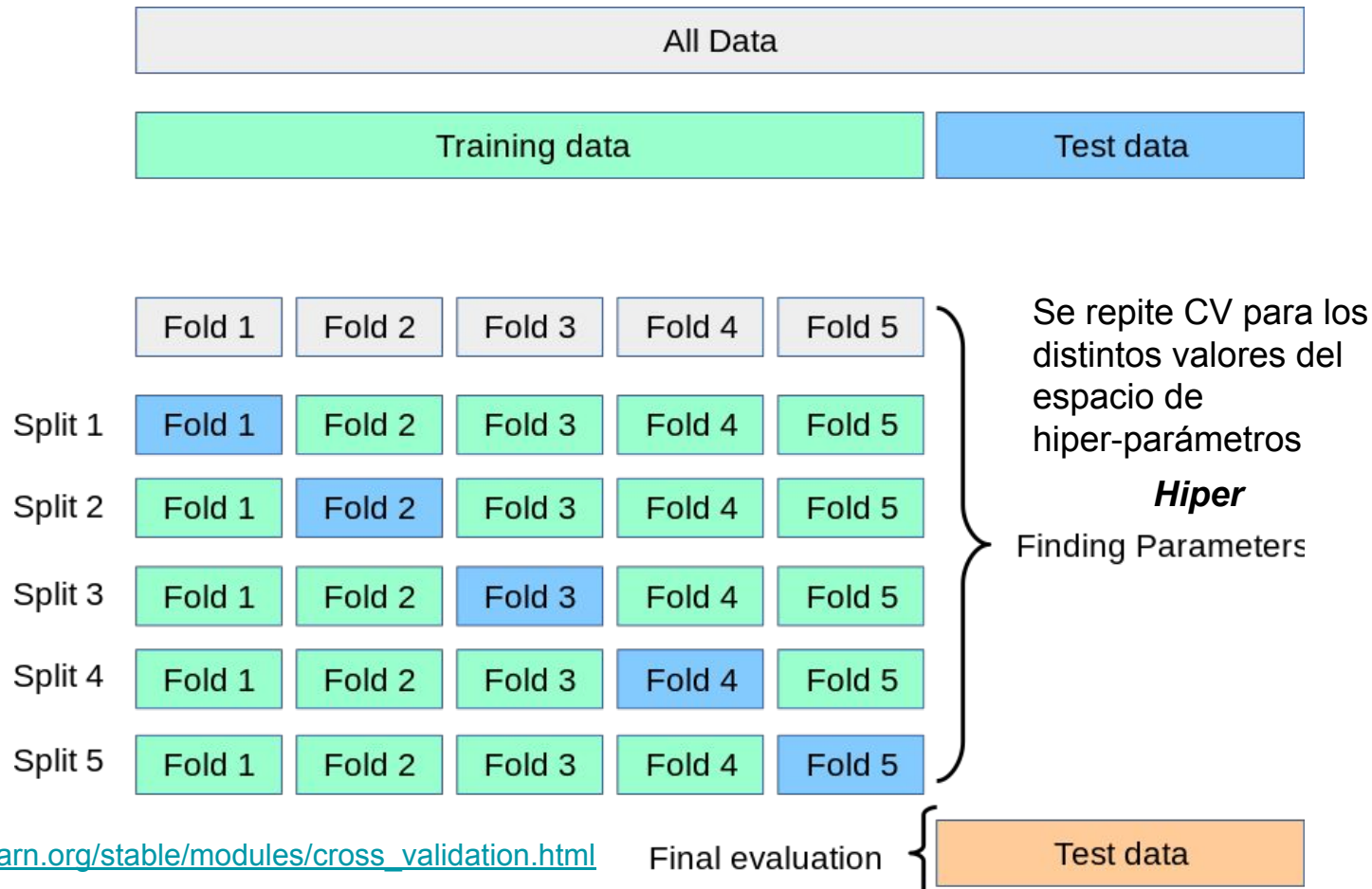
¡Una guía para ordenar el proceso!

¡Es lo que nosotros
llamamos
hiperparámetros!



https://scikit-learn.org/1.5/modules/cross_validation.html

Eligiendo el modelo y Evaluando el desempeño



`sklearn.model_selection.GridSearchCV`

```
class sklearn.model_selection.GridSearchCV(estimator, param_grid, scoring=None, n_jobs=None, iid='deprecated', refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs', error_score=nan, return_train_score=False)
```

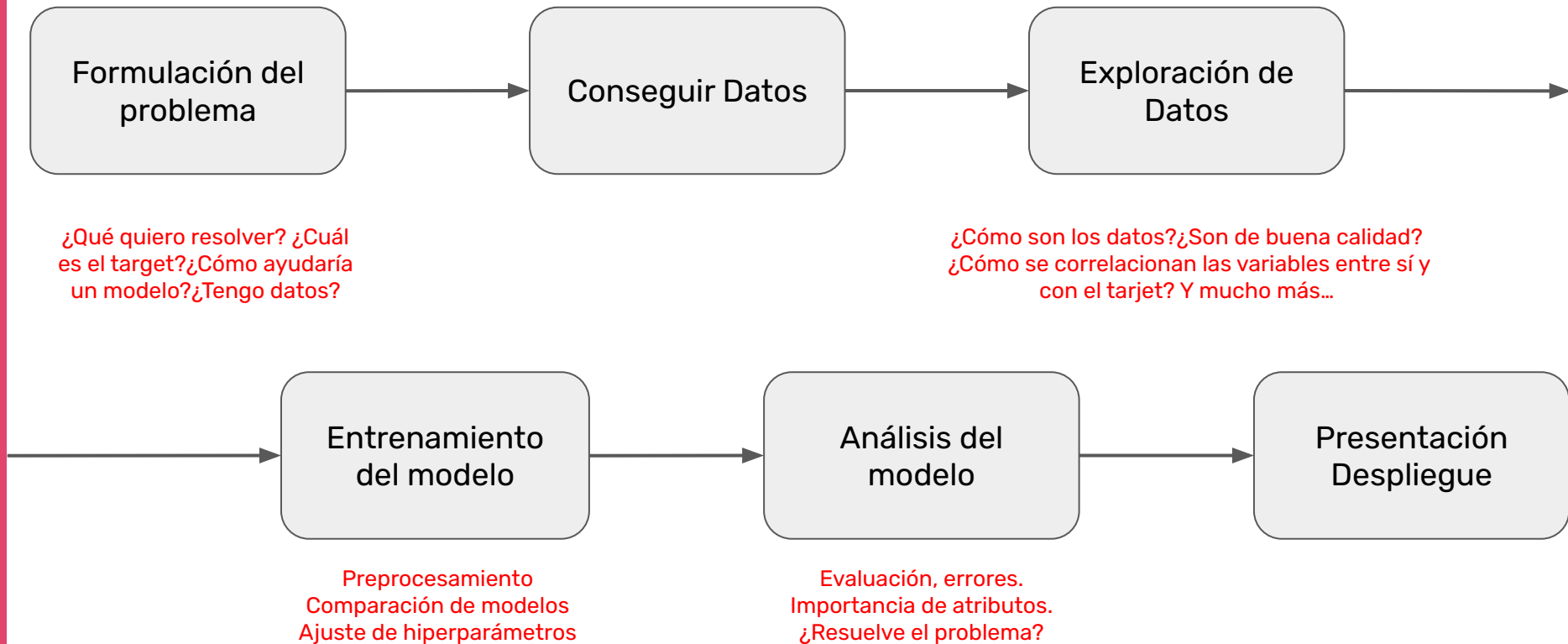
[\[source\]](#)

`sklearn.model_selection.RandomizedSearchCV`

```
class sklearn.model_selection.RandomizedSearchCV(estimator, param_distributions, n_iter=10, scoring=None, n_jobs=None, iid='deprecated', refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs', random_state=None, error_score=nan, return_train_score=False)
```

[\[source\]](#)

Ciclo de vida de un proyecto de Aprendizaje Automático



Ciclo de vida de un proyecto de Aprendizaje Automático

